Tutorial #2 - Tanks

Introduction

Welcome to the second of our AppGameKit Studio tutorials aimed at those of you who are just delving into the world of tier 1 (BASIC) coding. In this tutorial we'll be looking at a very simple shoot 'em up based on the classic game of Tanks. We'll be showing you a little more about sprites and developing a very simple AI system.

The entry level for these tutorials expects some initial understanding of the students in the use of AppGameKit Studio. For more beginner level tutorials they can read the AppGameKit Studio Online Guides and watch the YouTube Videos.

Students are also encouraged to join the Official AppGameKit Studio forums. There's a newbie section where new users will be helped by many experienced and friendly developers who use AppGameKit Studio all the time.

Teaching aids and objectives

These tutorials aim to teach coding and game development using a combination of example led and action-based teaching. You will learn from studying the code snippets, reading and understanding the comments and gain an additional understanding of each section through a number of simple exercises aimed at reinforcing each lesson and will help build your confidence by demonstrating the ease with which the code can be altered.

This second tutorial aims to introduce the concepts of basic coding and is designed to lay the groundwork of future developments. During this tutorial you will learn:

- More about variables
- Expand on Functions
- More about sprites, and introduce more ways to manipulate them
- How to use simple maths to create a basic AI

- Basic understandings of loading and playing sound effects

At any point, examples of almost all coding commands can be accessed from the IDE (integrated development environment) simple by either clicking directly on to a command or pressing F1 on the keyboard. You should be encouraged to learn this method from the beginning as it will prove to be an invaluable reference guide and is a resource often used even by coders who are very familiar with AppGameKit Studio.

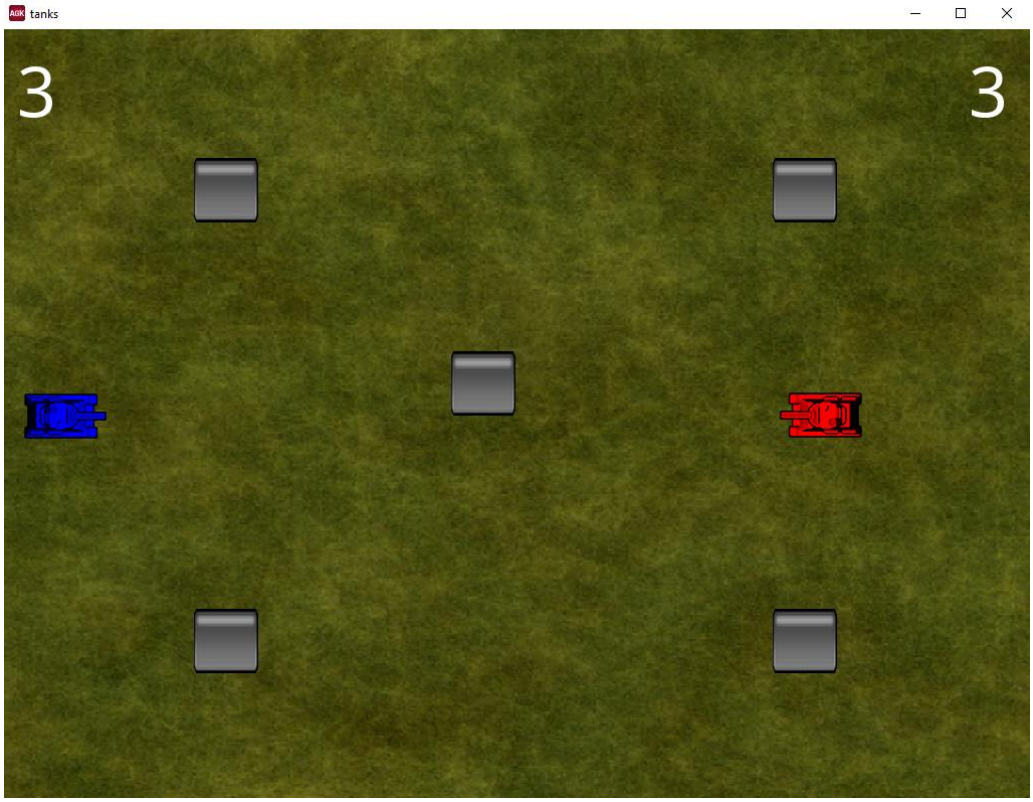
You can also access a full command reference from the IDE. By clicking Help on the top tool bar, you can gain access to a number of reference sources, including video tutorials, a list of useful keyboard shortcuts as well as the command reference.

Each section of code is available as a separate project which can be simply loaded into the IDE, but it it recommended that you learn to type the code into the IDE, as this will help to develop a familiarity with AppGameKit Studio overall and will promote confidence in using the functions of the IDE as well as reinforcing the learning process.

Let's get started

Due to the nature of this game, we're opening the full code at once rather than adding a few lines at a time. This is because the entire game is required to truly understand the individual elements in practice. So, let's look at the game. Launch AppGameKit Studio and open the project called *tanks.agk*

Compile and run the project and you should see the game start like this:



Currently the game is set up for one human player and one computer player.

Give the game a go, you can control the blue tanks using Z to rotate right, X to rotate left, S to move forward and Space to fire.

Once you've played a couple of games, let's take a look at the code.

At the very start of the code, you'll see the usual set up code.

```
// Project: tanks
// Created: 2017-04-18

// show all errors
SetErrorMode(2)

// set window properties
SetWindowTitle( "tanks" )
SetWindowSize( 1024, 768, 0 )
SetWindowAllowResize( 1 ) // allow the user to resize the window

// set display properties
SetVirtualResolution( 1024, 768 ) // doesn't have to match the window
SetOrientationAllowed( 1, 1, 1, 1 ) // allow both portrait and landscape on mobile devices
SetSyncRate( 30, 0 ) // 30fps instead of 60 to save battery
SetScissor( 0,0,0,0 ) // use the maximum available screen space, no black borders
UseNewDefaultFonts( 1 ) // since version 2.0.22 we can use nicer default fonts
```

You should be familar with all of this from the first tutorial, so, take a quick glance at the code to refresh your memory and we'll move on.

As for the pong tutorial, we set up the variables and sprites we need for our game at the start. Again, this code should be familar to you.

```
//Set up the player types
Type player
        //X and Y are the position on screen.
        x as float
        y as float
        //Which sprite will it use
        sprite as integer
        //We'll also need a special collison sprite
        colsprite as integer
        //In this game we move in the direction we are facing, so we'll
        //need to know what that is.
        rotation as float
        //Is the player controlled by the computer
        iscomputer as integer
        //Let's set up a variable to allow use to adjust the speed
        speed as float
        //How long since the tank last fired
        firerate as integer
        //And, for this game, we'll introduce some lives to give
        //everyone a few goes
        lives as integer
        //If the tanks is destroyed, we will spin it around
        //We need to know if it's spinning.
        spinning as integer
        //And for how long, so we will need a timer
        spintime as integer
        //For AI, we need to know if a collision has occured
        hascollided as integer
        //For AI, we'll need to know if the tank is turning around
        //An obstacle. We'll flag this
        isturning as integer
        //and where it is turning
        turnwhere as integer
EndType

//We only want 2 players, so we'll set up an array.
Dim tanks[2] as player

//Let's create a couple of text objects for the number of lives
```

```
CreateText(1,"0")
SetTextSize(1,90)
SetTextPosition(1,16,16)

CreateText(2,"0")
SetTextSize(2,90)
SetTextPosition(2,963,16)

//We'll need an image for our tank,
//Let's load it in.

LoadImage(1,"tank.png")

//We'll also need an image for our special collision sprite

LoadImage(2,"colsprite.png")

//Set up tanks fire sprites

Type tankfire
        //As tank fire, we need 2 sets of cordinates,
        //X and Y are the bullets current position
        x as float
        y as float
        //DX and DY are the destination.
        dx as float
        dy as float
        //Give it somewhere to store the sprite number
        sprite as integer
        //Is the bullet active?
        active as integer
        //How fast is it moving.
        speed as float
        //Who fired the bullet?
        //We'll need to know which player fired the bullet so we
        //don't shoot ourselves
        who as integer
EndType

//And now we set up the array.
//We'll allow 20, which is more than we'll need, but it will
//allow you to play around with fire rate and speed.
Dim tfire[20] as tankfire

//Let's set up some bullets.

//We only have 1 image to use, so let's load that in as image 2
LoadImage(3,"bullet.png")

For makebullets=1 to 20
        //give it a sprite number, starting at 100
        tfire[makebullets].sprite=99+makebullets
        //Make the sprite
        CreateSprite(tfire[makebullets].sprite,3)
        //Hide the sprites offscreen by moving them somewhere outside the screen
        //boundries
        SetSpritePosition(tfire[makebullets].sprite,-10000,-10000)
        //Set bullet depth to appear underneath the tanks
        SetSpriteDepth(tfire[makebullets].sprite,7)
        //We also want the bullets to be offset by the centre
        SetSpriteOffset(tfire[makebullets].sprite,4,7)
Next makebullets
```

Again, this code should be fairly familar. However, there are a couple of sprite commands that are worth exploring a little more.

```
SetSpriteDepth(tfire[makebullets].sprite,7)
```

When sprites are created, by default, they will all be drawn at a depth of 12. However, there are often times when we will want a sprite drawn in front of, or behind other sprites. For example, if you have a background, you would want all sprites drawn in front of it. In this case, we want our sprites drawn in front of our background, (which we'll load in a few moments), and behind our tanks, so we'll give each bullet sprite a depth of 7. The lower the sprites depth is set, the nearer the front

it will be drawn. So, a sprite depth set to 2 would be drawn in front of all sprites set 3 or higher.

The other sprite command worth looking at is:

```
SetSpriteOffset(tfire[makebullets].sprite,4,7)
```

When created, a sprite will be drawn and rotated from the top left-hand corner. This is often fine, but sometimes we will want to draw a sprite more centrally. As we want to rotate and draw all bullet sprites around the middle. The basic sprite size is 8 pixels wide and 15 pixels so its middle will be 4 pixels down, and 7.5 pixels in. We can't have half a pixel, so we'll call it 7.

Okay, now let's look at the code that sets up the player stats, and the function that does it.

```
//Now we've done that, we can set up the player stats
//So let's jump to our reset routine,
//This will allow us to set our player details up
//in the first place, as well as allow us to reset them between games.
//We can choose a full reset, restoring lives
//Or a partial reset, just resetting the tanks position and rotation.
//We'll use the value 0 for partial, and 1 for full
//For now, we'll want a full reset

resetplayers(1)

Function resetplayers(between)

        //First of all, let's make sure that any moving bullets are cancelled.

        For bullets=1 to 20
                tfire[bullets].active=0
                SetSpritePosition(tfire[bullets].sprite,-10000,-10000)
        Next bullets

        //Set up player 1
        tanks[1].x=64
        tanks[1].y=384
        tanks[1].sprite=1
        tanks[1].colsprite=2
        tanks[1].rotation=90
        tanks[1].iscomputer=0
        tanks[1].speed=4
        tanks[1].firerate=800

        If between=1
        tanks[1].lives=3
        EndIf

        //This is a special bit of code to create a tanks sprite if it doesn't exist.
        //This will only be set up once.

        //First we check if it exists.

        If GetSpriteExists(tanks[1].sprite)=0
                //If not make it
                CreateSprite(tanks[1].sprite,1)
                //We will also want to rotate the tank around it's middle
                //We can do this by setting an offset, so that it rotates around the centre
                SetSpriteOffset(tanks[1].sprite,24,42)
                //And the collision sprite
                CreateSprite(tanks[1].colsprite,2)
                //We don't need to see the colsprite, as we can still
                //Check collisions on non visible sprite.
                SetSpriteVisible(tanks[1].colsprite,0)
                //We will also want to rotate the colsprite around it's middle
                //We can do this by setting an offset, so that it rotates around the centre
                SetSpriteOffset(tanks[1].colsprite,16,16)

                //Place the tank at depth 5, so bullets will appear underneath for a better
                //Firing effect
                SetSpriteDepth(tanks[1].sprite,5)
                SetSpriteDepth(tanks[1].colsprite,5)
                //We also want the tanks to be different colours
```

```
            //We'll make the 1st one blue
            SetSpriteColor(tanks[1].sprite,0,0,255,255)
        EndIf

        //Set the tanks angle
        SetSpriteAngle(tanks[1].sprite,tanks[1].rotation)
        //Set it's position
        SetSpritePositionByOffset(tanks[1].sprite,tanks[1].x,tanks[1].y)


        //Set up player 2
        tanks[2].x=924
        tanks[2].y=384
        tanks[2].sprite=3
        tanks[2].colsprite=4
        tanks[2].rotation=270
        tanks[2].iscomputer=1
        tanks[2].speed=4
        tanks[2].firerate=800

        If between=1
        tanks[2].lives=3
        EndIf

        //This is a special bit of code to create a tanks sprite if it doesn't exist.
        //This will only be set up once.

        //First we check if it exists.

        If GetSpriteExists(tanks[2].sprite)=0
            //If not make it
            CreateSprite(tanks[2].sprite,1)
            //We will also want to rotate the tank around it's middle
            //We can do this by setting an offset, so that it rotates around the centre
            SetSpriteOffset(tanks[2].sprite,24,42)
            //And the collision sprite
            CreateSprite(tanks[2].colsprite,2)
            //We don't need to see the colsprite, as we can still
            //Check collisions on non visible sprite.
            SetSpriteVisible(tanks[2].colsprite,0)
            //We will also want to rotate the tank around it's middle
            //We can do this by setting an offset, so that it rotates around the centre
            SetSpriteOffset(tanks[2].colsprite,16,16)
            //Place the tank at depth 5, so bullets will appear underneath for a better
            //Firing effect
            SetSpriteDepth(tanks[2].sprite,5)
            SetSpriteDepth(tanks[2].colsprite,5)
            //We also want the tanks to be different colours
            //We'll make the 2nd one red
            SetSpriteColor(tanks[2].sprite,255,0,0,255)
        EndIf

        //Set the tanks angle
        SetSpriteAngle(tanks[2].sprite,tanks[2].rotation)
        //Set it's position
        SetSpritePositionByOffset(tanks[2].sprite,tanks[2].x,tanks[2].y)

    EndFunction
```

There are a couple of other Sprite commands being used here, you'll remember the use of `SetSpriteDepth`, notice how we have set the tank sprite and the collision sprite to 5. This will appear above the bullets, which if you recall, we set to depth 7.

EXERCISE: *Alter the speed of the blue (1st) tank. Try making it slower than the second tank and then faster. You'll only need to adjust a single variable to do this each time.*

Once you've played around with the speed of the tanks, reset them to 4 (The default) and let's try something else.

EXERCISE: *Try swapping the AI's, so the blue tank is controlled by the computer, and the red one by the player.*

The red tank is controlled by the arrow keys and Ctrl to fire.

Once you're happy with this, reset control to the default. (Red tank controlled by the computer and Blue by the player).

Now, let's create our background and wall sprites.

```
//Lastly, sprite wise, we'll need a background

LoadImage(4,"backdrop.png")
CreateSprite(5,4)

//Classic tank games have obstacles, so let's make some.
//We'll allow an obstacle every 64x64 pixels, which equals
//16 across, and 12 down
//Using this system we can easily create maps

//Firstly let's load an image

LoadImage(5,"wall.png")

Type bricks
//Which sprite number will it have
sprite as integer
//Is the wall active, we can create our own maps by turning a
//brink on or off.
active as integer
EndType

Dim obstacles[16,12] as bricks

nextsprite=200

//Let's make all the sprites
For across=1 to 16
        For down=1 to 12
                obstacles[across,down].sprite=nextsprite
                CreateSprite(nextsprite,5)
                SetSpritePosition(nextsprite,-64+(across*64),-64+(down*64))
                inc nextsprite,1
        Next down
Next across
```

We can have up to 16 walls across and 12 down. We obviously won't use them all everytime, but we will still want to create them. Notice how we position them using the values of *across* and *down*.

Now let's look at another new command and load in some sounds. Just like sprites and images, sounds have their own unique reference number, and just like sprites and images, a sound can share a reference with a sprite or image. This means you can have a sprite, an image and a sound all with the reference 1 (or any other number).

```
//We'll also be using sounds in this game
//These 2 sound effects are taken from the Sound Library DLC.
Global fire=1
LoadSound(fire,"fire.wav")
Global bang=2
LoadSound(bang,"bang.wav")
```

Ok, so we've created our sprites, images and sounds. Now we've just got a couple of other things to do. Firstly, we'll define a variable to hold the current time in milliseconds. You might recall that we do this, as calling the internal timer is pretty processor intensive and we want to call them as few times as we need to in a single game loop.

```
//We'll store the current time here
Global currenttime=0
```

```
We've just one last thing to do, and that's to set up our obstacles. Take a look

//You can create multiple levels
//Let's set up a variable to hold the current one.
Global level=1

makelevel(level)

We then call the Function makelevel(?), passing the number of the level we want to build.

Function makelevel(whichlevel)

        //Firstly, we need to deactivate all the sprites, so
        //We're only using the ones we need.
        For across=1 to 16
                For down=1 to 12
                        //Hide the sprite
                        SetSpriteVisible(obstacles[across,down].sprite,0)
                        //Make it inactive so it can't be detected
                        //For collisions.
                        SetSpriteActive(obstacles[across,down].sprite,0)
                        //Clear the map
                        obstacles[across,down].active=0
                Next down
        Next across

//Let's make a level
//All we need to do is set the square we want to have a block in
//As active by setting it to one.
//An easy way to make a map would be to plot it on graph paper
//More advanced users might try to make an editor.

//This is the first level

If whichlevel=1
        obstacles[4,3].active=1
        obstacles[13,3].active=1
        obstacles[4,10].active=1
        obstacles[13,10].active=1
        obstacles[8,6].active=1
EndIf

//Build the map

        For across=1 to 16
                For down=1 to 12
                        If obstacles[across,down].active=1
                                //Show the sprite
                                SetSpriteVisible(obstacles[across,down].sprite,1)
                                //Make it active so it can be detected
                                //For collisions.
                                SetSpriteActive(obstacles[across,down].sprite,1)
                        EndIf
                Next down
        Next across


EndFunction
```

For the tutorial we've only created a single level, but now it's time to have another go at something yourself. Take a look at the code above, and you'll see how we place obstacles on our grid. A good way of designing a level would be to mark out a 16x12 grid on some graph paper. So, if you wanted to place an obstacle in the top left corner you'd use the line:

```
obstacles[1,1].active=1
```

EXERCISE: *Have a go yourself and see what you can design. As a tip, try to keep some fairly large spaces as the AI doesn't use any advanced path finding techniques, we simply use a basic collision method for this tutorial to help keep things simple.*

Okay, now let's look at the main game loop.

```
//Main loop
do

     currenttime=GetMilliseconds()

    //Run through the players
    For loopthroughplayers=1 to 2
        SetTextString(loopthroughplayers,str(tanks[loopthroughplayers].lives))
        //We can only control the tank if it's not spinning
        If tanks[loopthroughplayers].spinning=0
                controlplayer(loopthroughplayers)
                Else
                //If it is spinning, let's do it!
                Inc tanks[loopthroughplayers].rotation,16
                //Set the angle

        SetSpriteAngle(tanks[loopthroughplayers].sprite,tanks[loopthroughplayers].rotation)
                //Only spin for 2 seconds.
                If currenttime-tanks[loopthroughplayers].spintime>2000
                        //Afterward, reset tank position
                        //If one of the players has run out of lives, reset everything
                        If tanks[loopthroughplayers].lives=0
                        resetplayers(1)
                        Else
                        resetplayers(0)
                        EndIf
                        //Stop the tanks spinning
                        tanks[loopthroughplayers].spinning=0

                EndIf
        EndIf
    Next loopthroughplayers

    //Move any bullets
    movefire()

    //Let's see if any bullet has hit a tank
    tankhit=checkhit()

    //Set the tank spinning
    If tankhit<>0
        //Play the bang sound effect
        PlaySound(bang)
        If tanks[tankhit].spinning=0
                tanks[tankhit].spinning=1
                tanks[tankhit].spintime=currenttime
        EndIf
    Endif


  //  Print( ScreenFPS() )

  // Update the screen.
    Sync()
loop
```

Most of this should be fairly clear to you if you've been paying attention. A couple of things are worth mentioning. One thing to notice is how we use the game loop to reference our text commands as both the tanks and the text can be reference using the values 1 or 2.

```
SetTextString(loopthroughplayers,str(tanks[loopthroughplayers].lives))
```

Also take a look at the STR command that we're using within the `SetTextString`. STR converts a numeric value into a String. We need to do this as text objects don't directly accept numeric values.

Now let's take a look at the control method. We call a Function called:

```
controlplayer(loopthroughplayers)
```

This Function is by far the largest in the game and is the most important. It controls player 1 and 2, either via the keyboard, or via some basic AI.

```
Function controlplayer(whichone)

//Set a couple of variables,
//turn is which way a tank is turning
//forward is if the tank needs to move forward
turn=0
forward=0

//If the player isn't the computer.
If tanks[whichone].iscomputer=0

        //Is it player 1?
        If whichone=1
                //Z turn left
                If GetRawKeyState(90)
                        turn=-1
                Endif

                //X turn right
                If GetRawKeyState(88)
                        turn=1
                Endif

                //S move forward
                If GetRawKeyState(83)
                        forward=1
                Endif

        //Space to fire, only 1 shot a second and only if the neither tank is spinning
        If GetRawKeyState(32) and currenttime-tanks[whichone].firerate>1000 and
        tanks[1].spinning=0 and tanks[2].spinning=0
        //Make fire
                makefire(whichone)
        Endif
Endif

        //Is it player 2?
        If whichone=2
        //Left arrow turn left
        If GetRawKeyState(37)
                turn=-1
        Endif

        //Right arrow turn right
        If GetRawKeyState(39)
                turn=1
        Endif

        //up arrow move forward
        If GetRawKeyState(38)
                forward=1
        Endif

        //Ctrl to fire, only 1 shot a second and only if the neither tank is spinning
        If GetRawKeyState(17) and currenttime-tanks[whichone].firerate>1000 and
        tanks[1].spinning=0 and tanks[2].spinning=0
        //Make fire
         makefire(whichone)
        Endif

        Endif

        EndIf

//If the player is the computer,
//Let's make it think
//Bear in mind that this is very simple AI
//Based on obstacle avoidance and opponent position
//There are more efficent, but more advanced techniques
If tanks[whichone].iscomputer=1
        //Find out which tanks we are looking at.
        otherone=1
        If whichone=1
                otherone=2
        Endif
```

```
//find the angle between the 2 tanks
        theangleis=FindAngle(tanks[whichone].x,tanks[whichone].y,tanks[otherone].x,tanks[othe
rone].y)

//If the other one is below the active tank,
//turn to face it.
        If theangleis<tanks[whichone].rotation
                turn=-2
        EndIf

        //If the other one is above the active tank,
        //turn to face it.
        If theangleis>tanks[whichone].rotation
                turn=2
        EndIf


        //When to move
        //Find the difference between the 2 angles
        //We use ABS to force negatives into positves as we
        //Only need the actual difference in degrees
        testangle=Abs(tanks[whichone].rotation-theangleis)

        //We also want to know how close the tanks are,
        //So they don't collide

        tankdistance#=GetDistance#(tanks[whichone].x,tanks[whichone].y,tanks[otherone].x,tank
s[otherone].y)

        //If the tank angle is far enough away, move tank
        If tankdistance#>100
                forward=1
        EndIf

        //Should the tank fire
        //For this we'll only allow an enemy shot every 2 seconds
        //And that the angle makes the shoot make sense
        //And lastly that it is within 500 pixels
        //And only if niether tank is spinning
        If currenttime-tanks[whichone].firerate>2000 and testangle<20 and tankdistance#<500
        and tanks[1].spinning=0 and tanks[2].spinning=0
        //Make fire
                makefire(whichone)
        Endif

        Endif

        //Now that we've decided where to move to
        //We can use the movement for both player and computer.

        //Now, let's make sure we can't leave the screen

        If tanks[whichone].x<64
                tanks[whichone].x=64
                forward=0
        EndIf

        If tanks[whichone].x>924
                tanks[whichone].x=924
                forward=0
        EndIf

        If tanks[whichone].y<64
                tanks[whichone].y=64
                forward=0
        EndIf

        If tanks[whichone].y>704
                tanks[whichone].y=704
                forward=0
        EndIf

        //Now we've decided on our ideal move, let's check for obstacles if the tank isn't
already avoiding one

        If tanks[whichone].isturning=0 and tanks[whichone].hascollided=0
                //Run through all the brick sprites
                For checkhits=200 to 361
```

```
                //Has the collision sprite hit a brick sprite
        If GetSpriteCollision(tanks[whichone].colsprite,checkhits) and
                GetSpriteActive(checkhits)

        //Check the position of the sprites against the position of the colsprite
        //We can then work out which direction is best to avoid the sprite
If  (GetSpriteX(checkhits)>=GetSpriteX(tanks[whichone].colsprite)) or
(GetSpriteY(checkhits)>=GetSpriteY(tanks[whichone].colsprite))
tanks[whichone].isturning=checkhits
tanks[whichone].turnwhere=4
Endif

//Check the position of the sprites against the position of the colsprite
//We can then work out which direction is best to avoid the sprite
If (GetSpriteX(checkhits)<GetSpriteX(tanks[whichone].colsprite)) or
(GetSpriteY(checkhits)<GetSpriteY(tanks[whichone].colsprite))
        tanks[whichone].isturning=checkhits
        tanks[whichone].turnwhere=-4
Endif

//End the loop if we have collided as we
//Only need to check one brick.
        checkhits=362
EndIf
Next checkhits
Endif


//If this collision is a human, then we only care if the tank
//Has actually hit something,
//If so, cancel automatic turning
//And prevent forward movement

If tanks[whichone].isturning<>0 and tanks[whichone].iscomputer=0
        tanks[whichone].isturning=0
        forward=0
EndIf

//If the tank is avoiding an obstacle, keep turning until colsprite is no longer
touching it.
If tanks[whichone].isturning>0
If GetSpriteCollision(tanks[whichone].colsprite,tanks[whichone].isturning) or
tanks[whichone].hascollided<>0
turn=tanks[whichone].turnwhere
//Stop the tank from moving
forward=0
//If the tank has stopped colliding, start a timer
ElseIf tanks[whichone].hascollided=0
tanks[whichone].hascollided=currenttime
EndIf
Endif


//If movement is automated,
//Keep turning for 1000 miliseconds after the tank has cleared the obstacle
//to ensure we make it through the gap.
If currenttime-tanks[whichone].hascollided>1000
        tanks[whichone].hascollided=0
        tanks[whichone].isturning=0
        tanks[whichone].turnwhere=0
Endif

If tanks[otherone].spinning<>0
        turn=0
        forward=0
Endif


If turn<>0 or forward<>0
        //If the tank is turning or moving let's find out what happens.
        //Set the rotation by adding the turn left or right to the current angle
        tanks[whichone].rotation=wrapangle(tanks[whichone].rotation+turn)
        SetSpriteAngle(tanks[whichone].sprite,tanks[whichone].rotation)

        //Set a position for the collison sprite
        tempx#=tanks[whichone].x+(sin(tanks[whichone].rotation)*32)
        tempy#=tanks[whichone].y-(cos(tanks[whichone].rotation)*32)
        SetSpritePositionByOffset(tanks[whichone].colsprite,tempx#,tempy#)

//If it's moving...
```

```
        If forward<>0
        //Using a little bit of math and we can move the sprite forward
        //towards it's direction of facing
        //Notice how we adjust the final move by the tanks speed.
        Inc tanks[whichone].x,sin(tanks[whichone].rotation)*tanks[whichone].speed
        Inc tanks[whichone].y,cos(tanks[whichone].rotation)*tanks[whichone].speed
        //Set the new position

        SetSpritePositionByOffset(tanks[whichone].sprite,tanks[whichone].x,tanks[whichone].y)

        EndIf

        Endif

EndFunction
```

Let's break this down into small sections. We'll start by looking at the keyboard controls. If you remember, we use scan codes to reference certain keys on the keyboard. Currently they are set as Z, X, S and Space for the blue tanks and the arrow keys and Ctrl for the red tank. We've chosen that layout because they are on different sides of the keyboard, making it easy for 2 human players to play the game at once.

Remember you can find a list of all the scancodes here:
https://www.appgamekit.com/documentation/guides/scancodes.htm

EXERCISE: *Change the keyboard controls for player 1 to something that suits a human vs computer layout.*

Once you've played around with the keyboard controls, let's take a look at the computer AI. As we've mentioned it's all pretty basic and all we're really checking for is basic turning to face the rival tank and some object avoidance based on a simple collision.

We have written three very useful Functions to speed up the development.

Firstly, *findangle* calculates the difference between two points in 2D space.

```
//This function calculates the difference between
//2 angles.
Function findangle(x#,y#,x1#,y1#)

        point1#=x#-x1#
        point2#=y#-y1#
        angle=ATanFull(point1#,point2#)-180
        angle=wrapangle(angle)

EndFunction angle
```

We're also using another Function called wrapangle(). This ensures that our angle never moves out of the range 0 – 360 degrees.

```
//This command will return a value that does not exceed the range of 0 to 360.
Function wrapangle( angle as float)
    local iChunkOut as integer
    local breakout as integer
    iChunkOut = angle
    iChunkOut = iChunkOut - mod( iChunkOut, 360 )
    angle = angle - iChunkOut
    breakout = 10000
        //Wrap the value
    while angle < 0.0 or angle >= 360.0
        if angle < 0.0 then angle = angle + 360.0
        if angle >= 360.0 then angle = angle - 360.0
        //Make sure we don't get caught in a loop
        //If we've made 1000 attempts and don't have a result
        //Exit the while - endwhile loop
```

```
        dec breakout
        if  breakout = 0  then exit
    endwhile
    if  breakout = 0  then angle = 0.0
endfunction angle
```

The last useful function we've written is to get the distance between two points in 2D space.

```
Function GetDistance#( X1#, Y1#, X2#, Y2#)

   Dist_X# = X1# - X2#
   Dist_Y# = Y1# - Y2#
   Distance# = Sqrt(Dist_X# * Dist_X# + Dist_Y# * Dist_Y#)

EndFunction Distance#
```

All three of these Functions use some simple maths to calculate the various results. We won't cover the maths here as we're teaching basic coding, but a strong background in maths will certainly help you with your own developments. You're of course free to use these Functions in your own games and apps and creating a solid library of useful Functions will speed up your development time greatly. Don't forget you can select any of the math commands and press F1 to get a description of their use.

Now let's break down the actual code behind the AI.

```
theangleis=FindAngle(tanks[whichone].x,tanks[whichone].y,tanks[othe
rone].x,tanks[otherone].y)
```

Finds the difference in angles between the two tanks.

We compare the angles with the current rotation of the tank in question, we then decide whether we need to turn left (a negative turn) or right (a positive turn) to face the enemy tank.

```
//If the other one is below the active tank,
//turn to face it.
If theangleis<tanks[whichone].rotation
       turn=-2
EndIf

//If the other one is above the active tank,
//turn to face it.

If theangleis>tanks[whichone].rotation
       turn=2
EndIf
```

Now we need to decide if we want to attempt to move towards the enemy tank. We do this by checking if we are more than 100 pixels away from the enemy tank, if so, we flag the tank to move forward.

```
//When to move
//Find the difference between the 2 angles
//We use ABS to force negatives into positves as we
//Only need the actual difference in degrees

testangle=Abs(tanks[whichone].rotation-theangleis)

//We also want to know how close the tanks are,
//So they don't collide

tankdistance#=GetDistance#(tanks[whichone].x,tanks[whichone].y,tanks[otherone].x,tanks[othero
ne].y)

//If the tank angle is far enough away, move tank
       If tankdistance#>100
              forward=1
```

```
        EndIf
```

Lastly, we decide if we should fire. We will fire at the enemy tank every two seconds and only when the tank is within 500 pixels. We also only fire if we are within 20 degrees of facing the enemy tank.

```
//Should the tank fire
//For this we'll only allow an enemy shot every 2 seconds
//And lastly that it is within 500 pixels
//And only if niether tank is spinning

If currenttime-tanks[whichone].firerate>2000 and testangle<20 and tankdistance#<500 and
tanks[1].spinning=0 and tanks[2].spinning=0

//Make fire
       makefire(whichone)
Endif
```

EXERCISE: *Try making the tank fire more often. Can you see how to make it happen?*

We also need to make sure that neither of the tanks can leave the screen. We do this by preventing the screen position being too near the edge of the screen and cancelling any forward movement.

```
//Now that we've decided where to move to
//We can use the movement for both player and computer.

//Now, let's make sure we can't leave the screen

If tanks[whichone].x<64
       tanks[whichone].x=64
       forward=0
EndIf

If tanks[whichone].x>924
       tanks[whichone].x=924
       forward=0
EndIf

If tanks[whichone].y<64
       tanks[whichone].y=64
       forward=0
EndIf

If tanks[whichone].y>704
       tanks[whichone].y=704
       forward=0
EndIf
```

Okay, now let's take a look at the final piece of the object avoidance code. Firstly we check to see if our collision sprite has hit an obstacle and that the obstacle is active.

If we do collide, we compare the position of the sprite with the position of the collision sprite. Depending on it's location we set the tank to turn left or right. This value is held in:

```
tanks[whichone].turnwhere
```

We only test until we've found a single collision and then jump out of the loop.

```
//Now we've decided on our ideal move, let's check for obstacles if the tank isn't already
avoiding one

If tanks[whichone].isturning=0 and tanks[whichone].hascollided=0
       //Run through all the brick sprites
       For checkhits=200 to 361
       //Has the collision sprite hit a brick sprite
       If GetSpriteCollision(tanks[whichone].colsprite,checkhits) and
GetSpriteActive(checkhits)
       //Check the position of the sprites against the position of the colsprite
       //We can then work out which direction is best to avoid the sprite
```

```
If  (GetSpriteX(checkhits)>=GetSpriteX(tanks[whichone].colsprite)) or
(GetSpriteY(checkhits)>=GetSpriteY(tanks[whichone].colsprite))
        tanks[whichone].isturning=checkhits
        tanks[whichone].turnwhere=4
Endif

//Check the position of the sprites against the position of the colsprite
//We can then work out which direction is best to avoid the sprite
If (GetSpriteX(checkhits)<GetSpriteX(tanks[whichone].colsprite)) or
(GetSpriteY(checkhits)<GetSpriteY(tanks[whichone].colsprite))
        tanks[whichone].isturning=checkhits
        tanks[whichone].turnwhere=-4
Endif

//End the loop if we have collided as we
//Only need to check one brick.
checkhits=362
EndIf
Next checkhits
Endif
```

We don't worry about the automatic turning if the tank is controlled by the player. All we care about is whether or not we can move forward.

```
//If this collision is a human, then we only care if the tank
//Has actually hit something,
//If so, cancel automatic turning
//And prevent forward movement

If tanks[whichone].isturning<>0 and tanks[whichone].iscomputer=0
        tanks[whichone].isturning=0
        forward=0
EndIf
```

Okay, so our tank is turning to avoid an obstacle. We will keep turning until the collision sprite is no longer in contact with an obstacle. We also cancel forward movement until we are clear of the object. Once we're clear of the obstacle, we start a timer to continue the movement for another second to ensure we have a clear path through.

```
//If the tank is avoiding an obstacle, keep turning until colsprite is no longer touching it.
If tanks[whichone].isturning>0
If GetSpriteCollision(tanks[whichone].colsprite,tanks[whichone].isturning) or
tanks[whichone].hascollided<>0
        turn=tanks[whichone].turnwhere
        //Stop the tank from moving
        forward=0
        //If the tank has stopped colliding, start a timer
        ElseIf tanks[whichone].hascollided=0
                tanks[whichone].hascollided=currenttime
        EndIf
Endif
```

As mentioned, we've finished our turning and a second has passed, so we can cancel all turning now.

```
//If movement is automated,
//Keep turning for 1000 miliseconds after the tank has cleared the obstacle
//to ensure we make it through the gap.
If currenttime-tanks[whichone].hascollided>1000
        tanks[whichone].hascollided=0
        tanks[whichone].isturning=0
        tanks[whichone].turnwhere=0
Endif
```

We will also cancel all turning and movement if the tank is spinning (been destroyed)

```
If tanks[otherone].spinning<>0
        turn=0
        forward=0
Endif
```

Lastly, we'll handle all of the turning and movement. This routine is shared by both human and computer controlled characters. We only bother if the tank is either moving, turning or both.

We also use a little bit of maths to ensure that the collision sprite is always facing forward of the tank as that direction is the only one we're interested in.

```
If turn<>0 or forward<>0
        //If the tank is turning or moving let's find out what happens.
        //Set the rotation by adding the turn left or right to the current angle
        tanks[whichone].rotation=wrapangle(tanks[whichone].rotation+turn)
        SetSpriteAngle(tanks[whichone].sprite,tanks[whichone].rotation)

        //Set a position for the collison sprite
        tempx#=tanks[whichone].x+(sin(tanks[whichone].rotation)*32)
        tempy#=tanks[whichone].y-(cos(tanks[whichone].rotation)*32)
        SetSpritePositionByOffset(tanks[whichone].colsprite,tempx#,tempy#)

        //If it's moving...
        If forward<>0
                //Using a little bit of math and we can move the sprite forward
                //towards it's direction of facing
                //Notice how we adjust the final move by the tanks speed.
                Inc tanks[whichone].x,sin(tanks[whichone].rotation)*tanks[whichone].speed
                Inc tanks[whichone].y,-cos(tanks[whichone].rotation)*tanks[whichone].speed
                //Set the new position

                SetSpritePositionByOffset(tanks[whichone].sprite,tanks[whichone].x,tanks[which
                one].y)

        EndIf

Endif
```

The final bit of code we'll be looking at for this tutorial is the fire Functions. For this we use four seperate Functions. `Makefire(?), movefire(), settankfiredest(x1,y1,x,y,number)` and `checkhit()`

We call makefire(), passing the number of the tank we want to fire from.

We firstly find a free bullet, if one is available, set the sprite position offset by the offsets we set when we first created our bullet sprites. We also rotate the sprite to face the direction of the tank, and thus the direction of facing.

We also flag the bullet as in use and by which tank (Tanks could be shoot by thier own bullet otherwise)

```
tanks[which].firerate=currenttime
//Set which tank the bullet belongs to,
//As we don't want to shoot ourselves.
tfire[freebullet].who=which
tfire[freebullet].active=1
```

We also decide where the bullet is heading using more maths.

```
tdestx#=tfire[freebullet].x+sin(tanks[which].rotation)*2000
tdesty#=tfire[freebullet].y-cos(tanks[which].rotation)*2000

settankfiredest(tfire[freebullet].x,tfire[freebullet].y,tdestx#,tdesty#,freebullet)
```

The full Function looks like this.

```
Function makefire(which)

freebullet=0

For find=1 to 20
        //Is the bullet free for use?
        if tfire[find].active=0
        //Set freebullet to the available space
        freebullet=find
        //By setting find to 21 (more than the end of the For-Next loop, we
        //can end the loop
        find=21
        EndIf
Next find


//Have we found a space
If freebullet<>0
        //Play the fire sound
        PlaySound(fire)
        //Set the last time the tank fired timer
        tanks[which].firerate=currenttime
        //Set which tank the bullet belongs to,
        //As we don't want to shoot ourselves.
        tfire[freebullet].who=which
        tfire[freebullet].active=1
        SetSpriteAngle(tfire[freebullet].sprite,tanks[which].rotation)
        tfire[freebullet].x=GetSpriteXByOffset(tanks[which].sprite)
        tfire[freebullet].y=GetSpriteYByOffset(tanks[which].sprite)
        tfire[freebullet].speed=16


        tdestx#=tfire[freebullet].x+sin(tanks[which].rotation)*2000
        tdesty#=tfire[freebullet].y-cos(tanks[which].rotation)*2000

        settankfiredest(tfire[freebullet].x,tfire[freebullet].y,tdestx#,tdesty#,freebullet)

        SetSpritePositionByOffset(tfire[freebullet].sprite,tfire[freebullet].x,tfire[freebull
et].y)

        Endif

EndFunction
```

Let's take a look at the `settankfiredest()` Function, this slight confusing name breaks down to mean Set Tank Fire Destination. We calculate this again using a little maths. We also pass the number of the bullet that we want to use. We need the start position and the proposed end position, the required movement that will need to be applied is returned and stored in the values `DX` and `DY`

```
Function settankfiredest(x1,y1,x,y,number)

originalX# = x1
originalY# = y1

 // work out the destination
 destinationX# = x
 destinationY# = y
   distanceX# = destinationX# - originalX#
   distanceY# = destinationY# - originalY#
    //We use a little maths to work out the distance
    distanceFromAtoB# = sqrt ( ( distanceX# * distanceX# ) + ( distanceY# * distanceY# ) )
       //From this we can calculate how the sprite needs to move to reach it's target
    if ( distanceFromAtoB# <> 0.0 )
       tfire[number].dx = distanceX# / distanceFromAtoB#
       tfire[number].dy = distanceY# / distanceFromAtoB#
    endif

  endfunction
```

Let's see the end result of this in the movement Function, `movefire()`

We will run through each bullet. If it's active, we will add the required movement to the bullet position, and increase it by the bullets speed. Once we've decided where to move the bullet to, all that's left is to set the sprites new position to the screen, using `SetSpritePositionByOffset.`

We will also check to see if the bullet has left the screen. If it has we will set it as non active.

```
Function movefire()

//Take a look at all the bullets
For movebullet=1 to 20
        //If the bullet is in use
        If tfire[movebullet].active=1
        //Move it
        Inc tfire[movebullet].x,tfire[movebullet].dx*tfire[movebullet].speed
        Inc tfire[movebullet].y,tfire[movebullet].dy*tfire[movebullet].speed
        //update the bullet sprite on screen

        SetSpritePositionByOffset(tfire[movebullet].sprite,tfire[movebullet].x,tfire[movebull
et].y)

        //Let's see if the bullet has left the screen

        If tfire[movebullet].x<-32 or tfire[movebullet].x>1056 or tfire[movebullet].y<-32 or
        tfire[movebullet].y>800
                tfire[movebullet].active=0
        EndIf
        EndIf
        //Look at the next one
        Next movebullet

EndFunction
```

All that remains to be done now is to check to see if any of these bullets that are flying around have hit anything. Once again, we'll run through all the bullets, if they are active, we'll check to see if it's collided with either an obstacle or a tank. If it's a tank we make sure that the player isn't shooting themselves. If they aren't it's an enemy bullet and we'll start the tank spinning. We check twice, once of each tank.

```
 Function checkhit()

        //Set up a variable to store
        //Which, if any tank has been hit
        hastankbeenhit=0

        //Loop through the tanks
        For whichtank=1 to 2
        //Loop through the bullets
        For whichbullet=1 to 20
                //Is the bullet active
                If tfire[whichbullet].active<>0
                        //Let's check to see if we hit a wall
                        //There are 192 wall sprites, numbered 200 - 391
                        For checkwall=200 to 391
                         //We're only interested in the active ones
                         If GetSpriteActive(checkwall)<>0
                                If GetSpriteCollision(tfire[whichbullet].sprite,checkwall)
                                tfire[whichbullet].active=0
                                //Disable it, making it available to be chosen again
                                //And hide it off the screen.
                                SetSpritePosition(tfire[whichbullet].sprite,-10000,-10000)
                                //If it's hit a wall, there no need to check if it hits a tank
                                //So we'll set the tfire[whichbullet].who to 0
                                tfire[whichbullet].who=0
                                Endif
                         Endif
                        Next checkwall
                //Make sure it doesn't belong to this tank
                //So we don't shoot ourselves
                If tfire[whichbullet].who<>whichtank
```

```
                //Check to see if the bullet sprite and the tank sprite collide
                If GetSpriteCollision(tfire[whichbullet].sprite,tanks[whichtank].sprite)
                        //Decrease the lives
                        Dec tanks[whichtank].lives,1
                        //If we've got a hit, flag the tank
                        hastankbeenhit=whichtank
                        //We don't want the bullet to contine
                        tfire[whichbullet].active=0
                        //Disable it, making it available to be chosen again
                        //And hide it off the screen.
                        SetSpritePosition(tfire[whichbullet].sprite,-10000,-10000)
                        //Only one bullet can hit a tank at a time,
                        //So we will force an end to our loop as we
                        //Don't need to continue
                        whichbullet=21
                        EndIf
                        Endif
                EndIf
        Next whichbullet
Next whichtank

//Return which tank, if any, has been hit.

EndFunction hastankbeenhit
```

EXERCISE: *Add a menu screen using everything you've learnt. As a tip, you could either use your own images, or create new text objects. You can read the keyboard to start a new game or exit the game.*

That's the end of this tutorial. We hope that you've learnt a lot more about coding. Why not play around with the code and see if you can add some other features. You already have everything you need to try to add a menu screen.